

Lecture 18: Orthogonalization (First part)

Instructors:

Christoph Lohrmann (christophl@ru.is)

Álfur Birkir Bjarnason (alfurb@ru.is)

[Based on slides for „Decoding the Matrix“]



WEEK

NINE



Agenda

Chapter 9

Lecture 18:

Fire Engine Problem
Project_orthogonal
Orthogonalization
Fire Engine Lemma
Vector decomposition





Lecture 17 - Summary

Theorems & Lemmas

Fire Engine Lemma

Projection of
b along a

Project_along

Pythagorean theorem

Concept & Definition

Inner Product

Vector Norm

Orthogonality

Previous

Solving Matrix
vector
equation

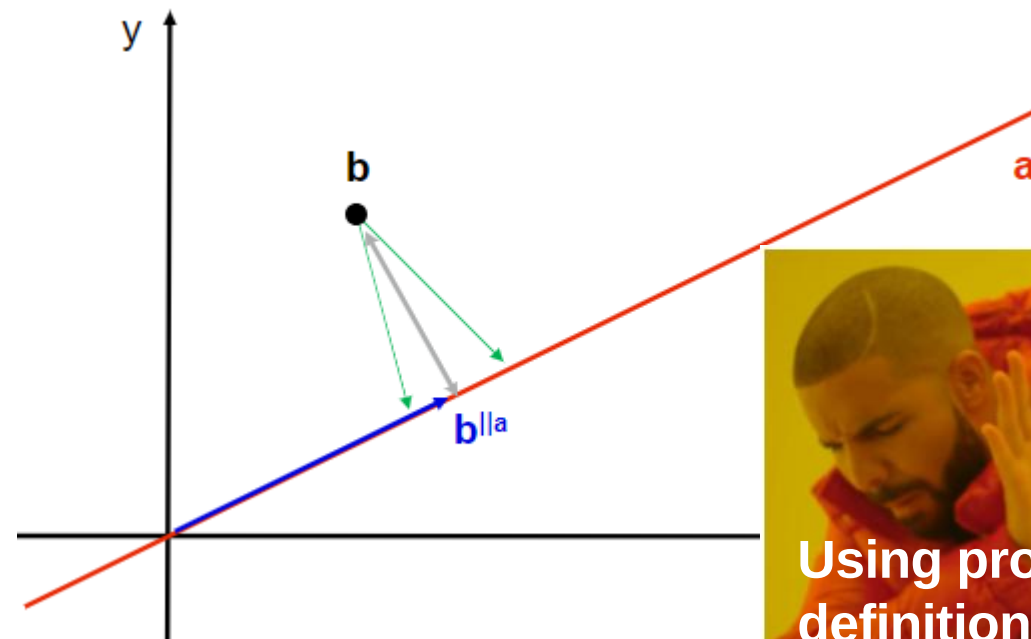
Tools



Lecture 18(+) - Goal

Previous 1D Fire Engine Problem

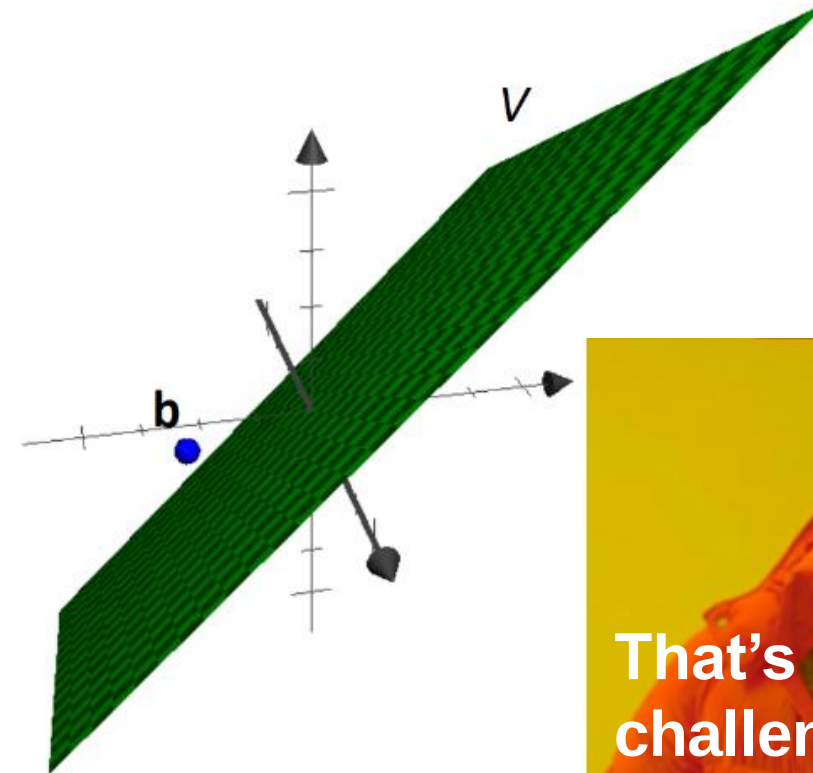
Previous goal: Find the closest point to $\mathbf{b}$ on the line $\mathbf{a}$.



Using projection
definition 8.3.6
and Lemma 8.3.11
too easy for me

Generalization of Fire Engine Problem

New goal: Find the closest point to $\mathbf{b}$ on the **plan** $\mathbf{V}$.



Now consider that the plan $\mathbf{V}$ is build by a list of vectors
 $\text{Span} \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$

That's a real
challenge for me
!



Closest point problem in higher dimension

Goal: An algorithm that, given a vector $\mathbf{b}$ and vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$, finds the vector $\mathbf{x}$ in $\text{Span} \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ that is closest to $\mathbf{b}$.

Let $A = \left[\begin{array}{c|c|c} \mathbf{v}_1 & \cdots & \mathbf{v}_n \end{array} \right]$.

Special case: We can use the algorithm to determine whether $\mathbf{b}$ lies in $\text{Span} \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$.

Minimization problem

$$\left\| \begin{bmatrix} \mathbf{b} \end{bmatrix} - \begin{bmatrix} \mathbf{v}_1 & \cdots & \mathbf{v}_n \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} \right\|$$

Using square norm

$$\left\| \begin{bmatrix} \mathbf{b} \end{bmatrix} - \begin{bmatrix} \mathbf{v}_1 & \cdots & \mathbf{v}_n \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} \right\|^2$$

Using row vector

$$\left\| \begin{bmatrix} \mathbf{b} \end{bmatrix} - \begin{bmatrix} \mathbf{a}_1 \\ \vdots \\ \mathbf{a}_m \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} \right\|^2$$

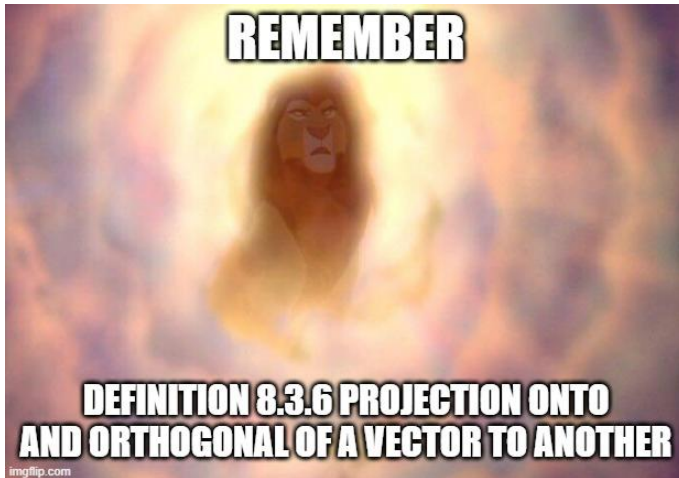


Much better

Find the vector $\mathbf{x}$ that minimizes $(\mathbf{b}[1] - \mathbf{a}_1 \cdot \mathbf{x})^2 + \dots + (\mathbf{b}[m] - \mathbf{a}_m \cdot \mathbf{x})^2$



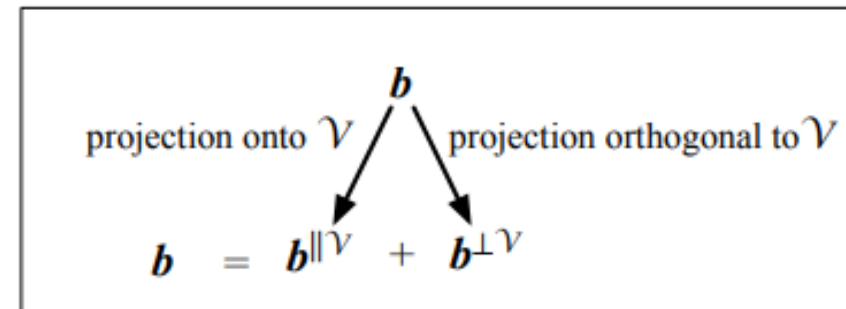
High dimensional projection onto/orthogonal to



Definition: For a vector $\mathbf{b}$ and a vector space $\mathcal{V}$, we define the projection of $\mathbf{b}$ onto $\mathcal{V}$ (written $\mathbf{b}^{\parallel \mathcal{V}}$) and the projection of $\mathbf{b}$ orthogonal to $\mathcal{V}$ (written $\mathbf{b}^{\perp \mathcal{V}}$) so that

$$\mathbf{b} = \mathbf{b}^{\parallel \mathcal{V}} + \mathbf{b}^{\perp \mathcal{V}}$$

and $\mathbf{b}^{\parallel \mathcal{V}}$ is in $\mathcal{V}$, and $\mathbf{b}^{\perp \mathcal{V}}$ is orthogonal to every vector in $\mathcal{V}$.



Extend 8.3.8 for vector space

High-Dimensional Fire Engine Lemma: The point in a vector space $\mathcal{V}$ closest to $\mathbf{b}$ is $\mathbf{b}^{\parallel \mathcal{V}}$ and the distance is $\|\mathbf{b}^{\perp \mathcal{V}}\|$.



Finding the projection of $\mathbf{b}$ orthogonal to $\text{Span}\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$

Finding the projection of $\mathbf{b}$ orthogonal to $\text{Span}\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$

Suppose V is specified by generators $\mathbf{v}_1, \dots, \mathbf{v}_n$

Goal: An algorithm for computing $\mathbf{b}^{\perp V}$ in this case.

- *input:* vector $\mathbf{b}$, vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$
- *output:* projection of $\mathbf{b}$ orthogonal to $V = \text{Span}\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$

We already know how to solve this when $n = 1$:

Let's try to generalize....



Project_orthogonal(b,vlist)

```
def project_orthogonal_1(b, v):  
    return b - project_along(b, v)
```



```
def project_orthogonal(b, vlist):  
    for v in vlist:  
        b = b - project_along(b, v)  
    return b
```





Project_orthogonal(b,vlist) doesn't work

```
def project_orthogonal(b, vlist):
    for v in vlist:
        b = b - project_along(b, v)
    return b
```

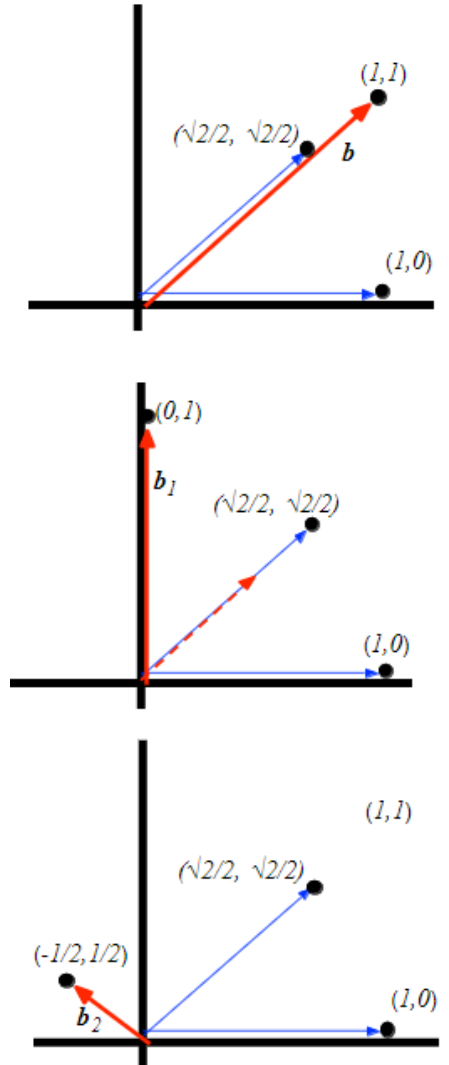
Example:

$$\mathbf{b} = [1, 1] \quad \text{vlist} = [[1, 0], \left[\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2} \right]]$$

Let $\mathbf{b}_i$ be value of the variable $\mathbf{b}$ after i iterations.

$$\begin{aligned} \mathbf{b}_1 &= \mathbf{b}_0 - (\text{projection of } [1, 1] \text{ along } [1, 0]) \\ &= \mathbf{b}_0 - [1, 0] \\ &= [0, 1] \end{aligned}$$

$$\begin{aligned} \mathbf{b}_2 &= \mathbf{b}_1 - (\text{projection of } [0, 1] \text{ along } \left[\frac{\sqrt{2}}{2}, \frac{\sqrt{2}}{2} \right]) \\ &= \mathbf{b}_1 - \left[\frac{1}{2}, \frac{1}{2} \right] \\ &= \left[-\frac{1}{2}, \frac{1}{2} \right] \text{ which is not orthogonal to } [1, 0] \end{aligned}$$





How to repair `project_orthogonal(b,vlist)`

```
def project_orthogonal(b, vlist):  
    for v in vlist:  
        b = b - project_along(b, v)  
    return b
```

➡ **Find : $w = b||v_1 + \dots + b||v_n$**

➡ **Then compute: $b - w$**

```
def classical_project_orthogonal(b, vlist):  
    w = all-zeroes-vector  
    for v in vlist:  
        w = w + project_along(b, v)  
    return b - w
```

➡ **Not working !** Final vector not orthogonal



What to do with `project_orthogonal(b,vlist)`

Example:

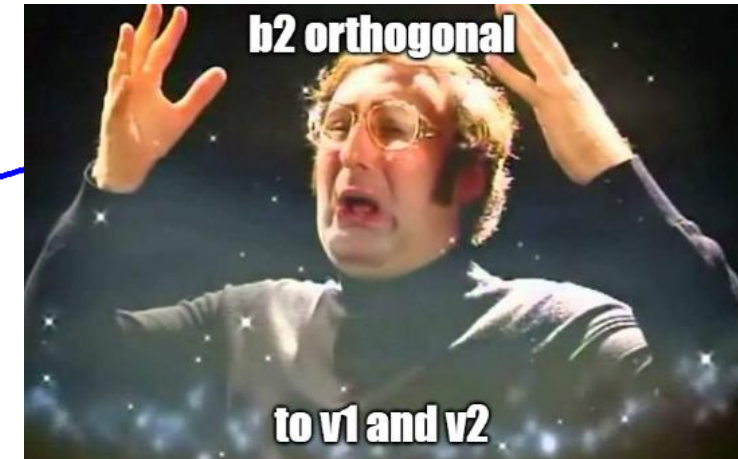
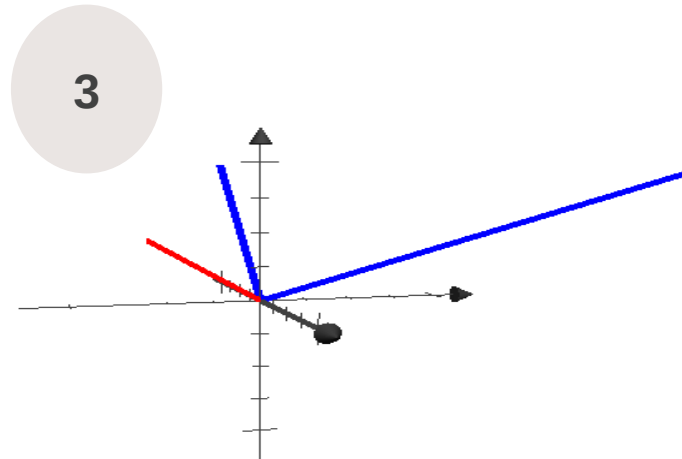
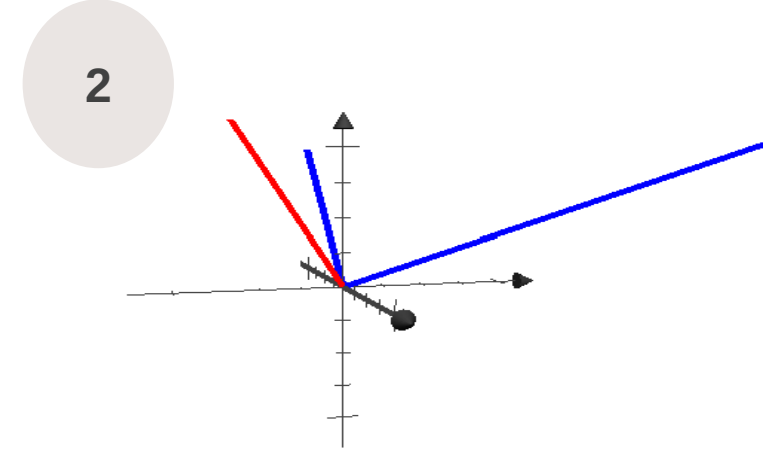
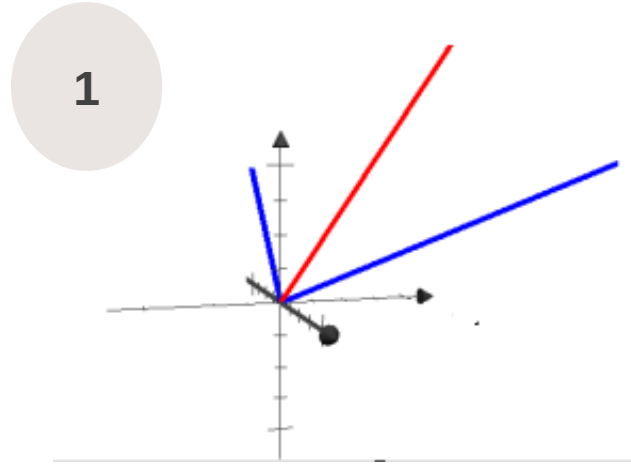
1 $\mathbf{b} = [1, 1, 2], \mathbf{v}_1 = [1, 2, 1] \mathbf{v}_2 = [-1, 2, -1]$

2
$$\mathbf{b}_1 = \mathbf{b}_0 - \frac{\langle \mathbf{b}_0, \mathbf{v}_1 \rangle}{\langle \mathbf{v}_1, \mathbf{v}_1 \rangle} \mathbf{v}_1$$

$$= \frac{1}{6}, -\frac{4}{6}, \frac{7}{6}$$

3
$$\mathbf{b}_2 = \mathbf{b}_1 - \frac{\langle \mathbf{b}_1, \mathbf{v}_2 \rangle}{\langle \mathbf{v}_2, \mathbf{v}_2 \rangle} \mathbf{v}_2$$

$$= \frac{2}{3}, -\frac{2}{3}, \frac{2}{3}$$





Maybe `project_orthogonal(b,vlist)` works with $\mathbf{v}_1$, $\mathbf{v}_2$ orthogonal ?

Assume $\langle \mathbf{v}_1, \mathbf{v}_2 \rangle = 0$.

First iteration:

$$\mathbf{b}_1 = \mathbf{b}_0 - \sigma_1 \mathbf{v}_1$$

Second iteration:

$$\mathbf{b}_2 = \mathbf{b}_1 - \sigma_2 \mathbf{v}_2$$

Check orthogonality between $\mathbf{v}_1$ and $\mathbf{b}_2$:

$$\begin{aligned} \langle \mathbf{v}_1, \mathbf{b}_2 \rangle &= \langle \mathbf{v}_1, \mathbf{b}_1 - \sigma_2 \mathbf{v}_2 \rangle \\ &= \langle \mathbf{v}_1, \mathbf{b}_1 \rangle - \langle \mathbf{v}_1, \sigma_2 \mathbf{v}_2 \rangle \\ &= \langle \mathbf{v}_1, \mathbf{b}_1 \rangle - \sigma_2 \langle \mathbf{v}_1, \mathbf{v}_2 \rangle \\ &= 0 + 0 \end{aligned}$$

Thus $\mathbf{b}_2$ is orthogonal to $\mathbf{v}_1$ and $\mathbf{v}_2$



Loop invariant of `project_orthogonal(b,vlist)`

New spec:

- *input*: a vector $\mathbf{b}$, and a list `vlist` of *mutually orthogonal* vectors
- *output*: the projection $\mathbf{b}^\perp$ of $\mathbf{b}$ orthogonal to the vectors in `vlist`

```
def project_orthogonal(b, vlist):
    for v in vlist:
        b = b - project_along(b, v)
    return b
```



Lemma 9.2.4 Loop invariant: Let `vlist` = $[\mathbf{v}_1, \dots, \mathbf{v}_n]$

For $i = 0, \dots, n$, let $\mathbf{b}_i$ be the value of the variable `b` after i iterations. Then $\mathbf{b}_i$ is the projection of $\mathbf{b}$ orthogonal to $\text{Span}\{\mathbf{v}_1, \dots, \mathbf{v}_i\}$. That is, we need to check for

Need to prove by induction
the invariant holds



Proof of loop invariant of `project_orthogonal(b, [v1, ..., vn])`

$\mathbf{b}_i$ = projection of $\mathbf{b}$ orthogonal to

for v in `vlist`:

`b = b - project_along(b, v)`

Must prove:

- ▶ $\mathbf{b}_k$ is orthogonal to $\mathbf{v}_1, \dots, \mathbf{v}_k$, ✓
- ▶ and $\mathbf{b} - \mathbf{b}_k$ is in $\text{Span} \{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ ✓

$\text{Span} \{\mathbf{v}_1, \dots, \mathbf{v}_i\}$:

- $\mathbf{b}_i$ is orthogonal to $\mathbf{v}_1, \dots, \mathbf{v}_i$, and
- $\mathbf{b} - \mathbf{b}_i$ is in $\text{Span} \{\mathbf{v}_1, \dots, \mathbf{v}_i\}$

Choice of σ_k ensures that $\mathbf{b}_k$ is orthogonal to $\mathbf{v}_k$.

Must show $\mathbf{b}_k$ also orthogonal to $\mathbf{v}_j$ for $j = 1, \dots, k - 1$

$$\begin{aligned} \langle \mathbf{b}_k, \mathbf{v}_j \rangle &= \langle \mathbf{b}_{k-1} - \sigma_k \mathbf{v}_k, \mathbf{v}_j \rangle \\ &= \langle \mathbf{b}_{k-1}, \mathbf{v}_j \rangle - \sigma_k \langle \mathbf{v}_k, \mathbf{v}_j \rangle \\ &= 0 - \sigma_k \langle \mathbf{v}_k, \mathbf{v}_j \rangle \\ &= 0 - \sigma_k 0 \end{aligned}$$

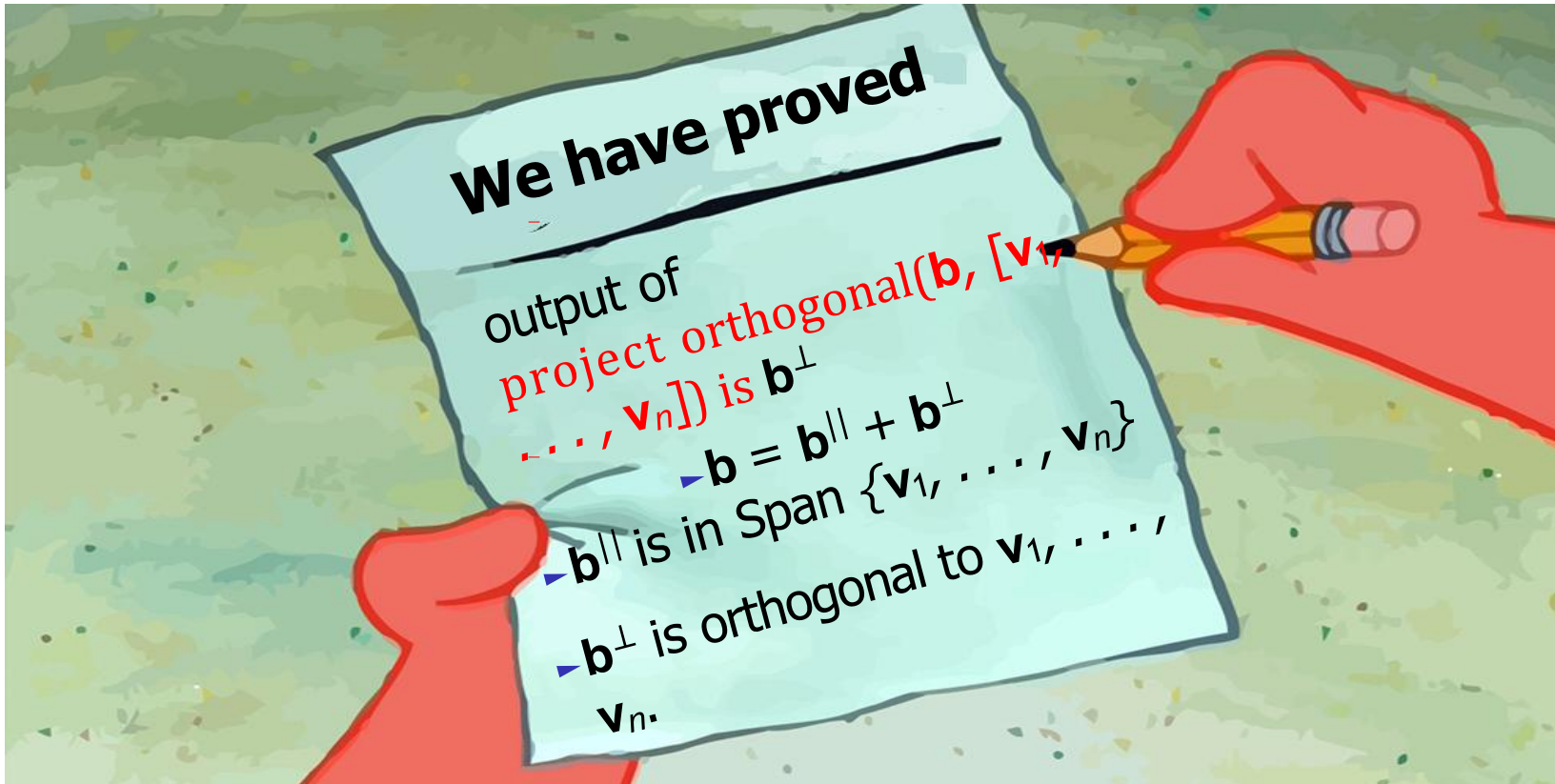
by the inductive hypothesis
by mutual orthogonality

Shows $\mathbf{b}_k$ orthogonal to $\mathbf{v}_1, \dots, \mathbf{v}_k$



Correctness of `project_orthogonal(b, vlist)`

```
def project_orthogonal(b, vlist):  
    for v in vlist:  
        b = b - project_along(b, v)  
    return b
```





Augmenting project orthogonal

```
def project_orthogonal(b, vlist):
    for v in vlist:
        b = b - project_along(b, v)
    return b
```

Next step:

- ▶ $\mathbf{v}_0, \dots, \mathbf{v}_n$
- ▶ *Augmented_project_orthogonal*

$$\mathbf{b} - \mathbf{b}^\perp = \alpha_0 \mathbf{v}_0 + \dots + \alpha_n \mathbf{v}_n$$

$$\mathbf{b} = \alpha_0 \mathbf{v}_0 + \dots + \alpha_n \mathbf{v}_n + 1 \mathbf{b}^\perp$$

$$\begin{bmatrix} \mathbf{b} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0 & \dots & \mathbf{v}_n & \mathbf{b}^\perp \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \vdots \\ \alpha_n \\ 1 \end{bmatrix}$$

Must create and populate a dictionary.

- ▶ One entry for each vector in *vlist*
- ▶ One additional entry, 1, for $\mathbf{b}^\perp$

Initialize dictionary with the additional entry.

```
def aug_project_orthogonal(b, vlist):
    alphadict = {len(vlist):1}
    for i in range(len(vlist)): v = vlist[i]
        sigma = (b*v)/(v*v) \
            if v*v > 0 else 0
        alphadict[i] = sigma
        b = b - sigma*v
    return (b, alphadict)
```



Building an orthogonal set of generators

Original stated goal:

Find the projection of $\mathbf{b}$ orthogonal to the space V spanned by arbitrary vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$.

New problem: Orthogonalization

- ▶ *input:* A list $[\mathbf{v}_1, \dots, \mathbf{v}_n]$ of vectors over the reals
- ▶ *output:* A list of mutually orthogonal vectors $\mathbf{v}_1^*, \dots, \mathbf{v}_n^*$ such that

$$\text{Span } \{\mathbf{v}_1^*, \dots, \mathbf{v}_n^*\} = \text{Span } \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$$



The orthogonalize procedure

Idea: Use `project_orthogonal` iteratively to make a longer and longer list of mutually orthogonal vectors.

- ▶ First consider $\mathbf{v}_1$. Define $\mathbf{v}_1^* := \mathbf{v}_1$ since the set $\{\mathbf{v}_1^*\}$ is trivially a set of mutually orthogonal vectors.
- ▶ Next, define $\mathbf{v}_2^*$ to be the projection of $\mathbf{v}_2$ orthogonal to $\mathbf{v}_1^*$.
- ▶ Now $\{\mathbf{v}_1^*, \mathbf{v}_2^*\}$ is a set of mutually orthogonal vectors.
- ▶ Next, define $\mathbf{v}_3^*$ to be the projection of $\mathbf{v}_3$ orthogonal to $\mathbf{v}_1^*$ and $\mathbf{v}_2^*$, so $\{\mathbf{v}_1^*, \mathbf{v}_2^*, \mathbf{v}_3^*\}$ is a set of mutually orthogonal vectors....

In each step, we use `project_orthogonal` to find the next orthogonal vector.

In the i^{th} iteration, we project $\mathbf{v}_i$ orthogonal to $\mathbf{v}_1^*, \dots, \mathbf{v}_{i-1}^*$ to find $\mathbf{v}_i^*$.

```
def orthogonalize(vlist):  
    vstarlist = []  
    for v in vlist:  
        vstarlist.append(project_orthogonal(v,  
            vstarlist))  
    return vstarlist
```



Correctness of the orthogonalize procedure, Part I

```
def orthogonalize(vlist):  
    vstarlist = []  
    for v in vlist:  
        vstarlist.append(project_orthogonal(v,  
            vstarlist))  
    return vstarlist
```

Lemma: Throughout the execution of **orthogonalize**, the vectors in **vstarlist** are mutually orthogonal.

Proof:



Example of orthogonalize

Example: **orthogonalize** ($\{\mathbf{v}_1 = [2, 0, 0], \mathbf{v}_2 = [1, 2, 2], \mathbf{v}_3 = [1, 0, 2]\}$)

it returns the list `vstarlist` consisting of

$$\mathbf{v}_1^* = [2, 0, 0], \mathbf{v}_2^* = [0, 2, 2], \mathbf{v}_3^* = [0, -1, 1]$$

- (1) In the first iteration, when v is $\mathbf{v}_1$, `vstarlist` is empty, so the first vector $\mathbf{v}_1^*$ added to `vstarlist` is $\mathbf{v}_1$ itself.
- (2) In the second iteration, when v is $\mathbf{v}_2$, `vstarlist` consists only of $\mathbf{v}_1^*$. The projection of $\mathbf{v}_2$ orthogonal to $\mathbf{v}_1^*$ is

$$\begin{aligned} \mathbf{v}_2 - \frac{\langle \mathbf{v}_2, \mathbf{v}_1^* \rangle}{\langle \mathbf{v}_1^*, \mathbf{v}_1^* \rangle} \mathbf{v}_1^* &= [1, 2, 2] - \frac{2}{4} [2, 0, 0] \\ &= [0, 2, 2] \end{aligned}$$

so $\mathbf{v}_2^* = [0, 2, 2]$ is added to `vstarlist`.

- (3) In the third iteration, when v is $\mathbf{v}_3$, `vstarlist` consists of $\mathbf{v}_1^*$ and $\mathbf{v}_2^*$. The projection of $\mathbf{v}_3$ orthogonal to $\mathbf{v}_1^*$ is $[0, 0, 2]$, and the projection of $[0, 0, 2]$ orthogonal to $\mathbf{v}_2^*$ is

$$[0, 0, 2] - \frac{1}{2} [0, 2, 2] = [0, -1, 1]$$

so $\mathbf{v}_3^* = [0, -1, 1]$ is added to `vstarlist`



Correctness of the orthogonalize procedure, Part II

Lemma: Consider **orthogonalize** applied to an n -element list $[\mathbf{v}_1, \dots, \mathbf{v}_n]$.

After i iterations of the algorithm, $\text{Span } \mathbf{vstarlist} = \text{Span } \{\mathbf{v}_1, \dots, \mathbf{v}_i\}$.

Proof: by induction on i .

- $i=0$, is trivial.
- For $i-1$, $\mathbf{vstarlist} = \text{Span } \{\mathbf{v}_1^*, \dots, \mathbf{v}_{i-1}^*\}$, so

$$\text{Span } \{\mathbf{v}_1^*, \dots, \mathbf{v}_{i-1}^*\} = \text{Span } \{\mathbf{v}_1, \dots, \mathbf{v}_{i-1}\}$$

by adding $\mathbf{v}_i$ need to demonstrate this hold:

$$\text{Span } \{\mathbf{v}_1^*, \dots, \mathbf{v}_{i-1}^*, \mathbf{v}_i\} = \text{Span } \{\mathbf{v}_1, \dots, \mathbf{v}_{i-1}, \mathbf{v}_i\}$$

From **project orthogonal** $(\mathbf{v}_i, [\mathbf{v}_1^*, \dots, \mathbf{v}_{i-1}^*])$, we have:

$$\mathbf{v}_i = \alpha_1 \mathbf{v}_1^* + \dots + \alpha_{i-1,i} \mathbf{v}_{i-1}^* + \mathbf{v}_i^*$$



Matrix form for orthogonalize

For **project orthogonal**, we had

$$\begin{bmatrix} \mathbf{b} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0 & \cdots & \mathbf{v}_n & \mathbf{b}^\perp \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \vdots \\ \alpha_n \\ 1 \end{bmatrix}$$

For **orthogonalize**, we have

$$\begin{bmatrix} \mathbf{v}_0 \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0^* \end{bmatrix} \begin{bmatrix} 1 \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{v}_1 \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0^* & \mathbf{v}_1^* \end{bmatrix} \begin{bmatrix} \alpha_{01} \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0^* & \mathbf{v}_1^* & \mathbf{v}_2^* \end{bmatrix} \begin{bmatrix} \alpha_{02} \\ \alpha_{12} \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} \mathbf{v}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0^* & \mathbf{v}_1^* & \mathbf{v}_2^* & \mathbf{v}_3^* \end{bmatrix} \begin{bmatrix} \alpha_{03} \\ \alpha_{13} \\ \alpha_{23} \\ 1 \end{bmatrix}$$



Example of matrix form for orthogonalize

Example: vlist :

$$\mathbf{v}_0 = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \mathbf{v}_1 = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$$

Vstarlist:

$$\mathbf{v}_0^* = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}, \mathbf{v}_1^* = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix}, \mathbf{v}_2^* = \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

Corresponding matrix equation:

$$\left[\begin{array}{c|c|c} \mathbf{v}_0 & \mathbf{v}_1 & \mathbf{v}_2 \end{array} \right] = \left[\begin{array}{c|c|c} 2 & 0 & 0 \\ 0 & 2 & -1 \\ 0 & 2 & 1 \end{array} \right] \left[\begin{array}{ccc} 1 & 0.5 & 0.5 \\ & 1 & 0.5 \\ & & 1 \end{array} \right]$$



Solving the closest point in the span of many vectors

Let $V = \text{Span} \{\mathbf{v}_0, \dots, \mathbf{v}_n\}$.

$\mathbf{b}^{\parallel V}$ solution, which is $\mathbf{b} - \mathbf{b}^{\perp V}$.

Two solutions to find $\mathbf{b}^{\perp V}$,

► *One method:*

Step 1: Apply **orthogonalize** to $\mathbf{v}_0, \dots, \mathbf{v}_n$ and obtain $\mathbf{v}_0^*, \dots, \mathbf{v}_n^*$
(Now $V = \text{Span} \{\mathbf{v}_0^*, \dots, \mathbf{v}_n^*\}$)

Step 2: Call **project_orthogonal**($\mathbf{b}, [\mathbf{v}_0^*, \dots, \mathbf{v}_n^*]$)
and obtain $\mathbf{b}^{\perp}$ as the result.

► *Another method:* Use **orthogonalize** ($[\mathbf{v}_0, \dots, \mathbf{v}_n, \mathbf{b}]$) to obtain $[\mathbf{v}_0^*, \dots, \mathbf{v}_n^*, \mathbf{b}^*]$.



Mutually orthogonal nonzero vectors are linearly independent

Proposition: Mutually orthogonal nonzero vectors are linearly independent.

Proof: Let $\mathbf{v}_0^*, \mathbf{v}_1^*, \dots, \mathbf{v}_n^*$ be mutually orthogonal nonzero vectors. Suppose $\alpha_0, \alpha_1, \dots, \alpha_n$ are coefficients such that

$$\mathbf{0} = \alpha_0 \mathbf{v}_0^* + \alpha_1 \mathbf{v}_1^* + \dots + \alpha_n \mathbf{v}_n^*$$

Must show that $\alpha_0 = \alpha_1 = \dots = \alpha_n = 0$.

Computes :

$$\begin{aligned} \langle \mathbf{v}_0^*, \mathbf{0} \rangle &= \langle \mathbf{v}_0^*, \alpha_0 \mathbf{v}_0^* + \alpha_1 \mathbf{v}_1^* + \dots + \alpha_n \mathbf{v}_n^* \rangle \\ &= \alpha_0 \langle \mathbf{v}_0^*, \mathbf{v}_0^* \rangle + \dots + \alpha_n \langle \mathbf{v}_0^*, \mathbf{v}_n^* \rangle \\ &= \alpha_0 \langle \mathbf{v}_0^*, \mathbf{v}_0^* \rangle + \dots + \alpha_n \langle \mathbf{v}_0^*, \mathbf{v}_n^* \rangle \\ &= \alpha_0 \|\mathbf{v}_0^*\|^2 + \alpha_1 0 + \dots + \alpha_n 0 \\ &= \alpha_0 \|\mathbf{v}_0^*\|^2 \end{aligned}$$



Computing basis

Proposition: Mutually orthogonal nonzero vectors are linearly independent.

What happens if we call the `orthogonalize` procedure on a list `vlist=[ $\mathbf{v}_0, \dots, \mathbf{v}_n$ ]` of vectors that are linearly dependent?

$\dim \text{Span } \{\mathbf{v}_0, \dots, \mathbf{v}_n\} < n + 1.$

`orthogonalize([ $\mathbf{v}_0, \dots, \mathbf{v}_n$ ])` returns $[\mathbf{v}_0^*, \dots, \mathbf{v}_n^*]$

The vectors $\mathbf{v}_0^*, \dots, \mathbf{v}_n^*$ are mutually orthogonal.

They can't be linearly independent since they span a space of dimension less than $n + 1$.

Therefore some of them must be zero vectors.

Leaving out the zero vectors does not change the space spanned...

Let S be the subset of $\{\mathbf{v}_0^*, \dots, \mathbf{v}_n^*\}$ consisting of nonzero vectors.

$\text{Span } S = \text{Span } \{\mathbf{v}_0^*, \dots, \mathbf{v}_n^*\} = \text{Span } \{\mathbf{v}_0, \dots, \mathbf{v}_n\}$

Proposition implies that S is linearly independent.

Thus S is a basis for $\text{Span } \{\mathbf{v}_0, \dots, \mathbf{v}_n\}$.



Computing basis

Therefore in principle the following algorithm computes a basis for $\text{Span } \{\mathbf{v}_0, \dots, \mathbf{v}_n\}$:

```
def find_basis( $[\mathbf{v}_0, \dots, \mathbf{v}_n]$ ):  
    "Return the list of nonzero starred vectors."  
     $[\mathbf{v}_0^*, \dots, \mathbf{v}_n^*] = \text{orthogonalize}([\mathbf{v}_0, \dots, \mathbf{v}_n])$   
    return  $[\mathbf{v}^* \text{ for } \mathbf{v}^* \text{ in } [\mathbf{v}_0^*, \dots, \mathbf{v}_n^*] \text{ if } \mathbf{v}^* \text{ is not the zero vector}]$ 
```

Recall

Lemma: Every finite set T of vectors contains a subset S that is a basis for $\text{Span } T$.

What about finding a subset of $\mathbf{v}_0, \dots, \mathbf{v}_n$ that is a basis?

Proposed algorithm:

```
def find_subset_basis( $[\mathbf{v}_0, \dots, \mathbf{v}_n]$ ):  
    "Return the list of original vectors that correspond to nonzero starred vectors."  
     $[\mathbf{v}_0^*, \dots, \mathbf{v}_n^*] = \text{orthogonalize}([\mathbf{v}_0, \dots, \mathbf{v}_n])$   
    Return  $[\mathbf{v}_i \text{ for } i \text{ in } \{0, \dots, n\} \text{ if } \mathbf{v}_i^* \text{ is not the zero vector}]$ 
```



Correctness of find_subset_basis

```
def find_subset_basis([ $\mathbf{v}_0, \dots, \mathbf{v}_n$ ]):
    [ $\mathbf{v}_0^*, \dots, \mathbf{v}_n^*$ ] = orthogonalize([ $\mathbf{v}_0, \dots, \mathbf{v}_n$ ])
    Return [ $\mathbf{v}_i$  for  $i$  in  $\{0, \dots, n\}$  if  $\mathbf{v}_i^*$  is not
           the zero vector]

def orthogonalize(vlist):
    vstarlist = []
    for v in vlist:
        vstarlist.append(
            project_orthogonal(v, vstarlist))
    return vstarlist
```

Example: `orthogonalize([ $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \mathbf{v}_4, \mathbf{v}_5, \mathbf{v}_6$ ])` returns [$\mathbf{v}_0^*, \mathbf{v}_1^*, \mathbf{v}_2^*, \mathbf{v}_3^*, \mathbf{v}_4^*, \mathbf{v}_5^*, \mathbf{v}_6^*$]

Suppose $\mathbf{v}_2^*$, $\mathbf{v}_4^*$, and $\mathbf{v}_5^*$ are zero vectors.

In iteration 3 iteration of `orthogonalize`, `project_orthogonal( $\mathbf{v}_3$ , [ $\mathbf{v}_0^*, \mathbf{v}_1^*, \mathbf{v}_2^*$ ])` computes $\mathbf{v}_3^*$:

- ▶ subtract projection of $\mathbf{v}_3$ along $\mathbf{v}_0^*$,
- ▶ subtract projection along $\mathbf{v}_1^*$,
- ▶ subtract projection along $\mathbf{v}_2^*$ —but since $\mathbf{v}_2^* = \mathbf{0}$, the projection is the zero vector

Result is the same as `project_orthogonal( $\mathbf{v}_3$ , [ $\mathbf{v}_0^*, \mathbf{v}_1^*$ ])`. Zero starred vectors are ignored.

Thus `orthogonalize([ $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_3, \mathbf{v}_6$ ])` would return [$\mathbf{v}_0^*, \mathbf{v}_1^*, \mathbf{v}_3^*, \mathbf{v}_6^*$].

Since [$\mathbf{v}_0^*, \mathbf{v}_1^*, \mathbf{v}_3^*, \mathbf{v}_6^*$] is a basis for $\mathcal{V} = \text{Span} \{ \mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \mathbf{v}_4, \mathbf{v}_5, \mathbf{v}_6 \}$ and [$\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_3, \mathbf{v}_6$] spans the same space, and has the same cardinality [$\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_3, \mathbf{v}_6$] is also a basis for $\mathcal{V}$.



Algorithm for finding basis for null space

Now let's find null space of matrix with columns $\mathbf{v}_1, \dots, \mathbf{v}_n$.

$$\begin{bmatrix} \mathbf{v}_0 & \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} = \begin{bmatrix} \mathbf{v}_0^* & \mathbf{v}_1^* & \mathbf{v}_2^* & \cdots & \mathbf{v}_n^* \end{bmatrix} \begin{bmatrix} 1 & \alpha_{01} & \alpha_{02} & & \alpha_{0n} \\ & 1 & \alpha_{12} & & \alpha_{1n} \\ & & 1 & & \alpha_{2n} \\ & & & \ddots & \\ & & & & 1 \end{bmatrix}$$

Can transform this to express starred vectors in terms of original vectors.

$$\begin{bmatrix} \mathbf{v}_0 & \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \end{bmatrix} \begin{bmatrix} 1 & \alpha_{01} & \alpha_{02} & & \alpha_{0n} \\ & 1 & \alpha_{12} & & \alpha_{1n} \\ & & 1 & & \alpha_{2n} \\ & & & \ddots & \\ & & & & 1 \end{bmatrix}^{-1} = \begin{bmatrix} \mathbf{v}_0^* & \mathbf{v}_1^* & \mathbf{v}_2^* & \cdots & \mathbf{v}_n^* \end{bmatrix}$$



Basis for null space

$$\begin{aligned}
 & \left[\begin{array}{c|c|c|c|c|c|c} \mathbf{v}_0 & \mathbf{v}_1 & \mathbf{v}_2 & \mathbf{v}_3 & \mathbf{v}_4 & \mathbf{v}_5 & \mathbf{v}_6 \end{array} \right] \begin{bmatrix} 1 & \alpha_{01} & \alpha_{02} & \alpha_{03} & \alpha_{04} & \alpha_{05} & \alpha_{06} \\ & 1 & \alpha_{12} & \alpha_{13} & \alpha_{14} & \alpha_{15} & \alpha_{16} \\ & & 1 & \alpha_{23} & \alpha_{24} & \alpha_{25} & \alpha_{26} \\ & & & 1 & \alpha_{34} & \alpha_{35} & \alpha_{36} \\ & & & & 1 & \alpha_{45} & \alpha_{46} \\ & & & & & 1 & \alpha_{56} \\ & & & & & & 1 \end{bmatrix}^{-1} \\
 &= \left[\begin{array}{c|c|c|c|c|c|c} \mathbf{v}_0^* & \mathbf{v}_1^* & \mathbf{v}_2^* & \mathbf{v}_3^* & \mathbf{v}_4^* & \mathbf{v}_5^* & \mathbf{v}_6^* \end{array} \right]
 \end{aligned}$$

Suppose $\mathbf{v}_2^*$, $\mathbf{v}_4^*$, and $\mathbf{v}_5^*$ are (approximately) zero vectors.

- ▶ Corresponding columns of inverse triangular matrix are nonzero vectors of the null space of the leftmost matrix.
- ▶ These columns are clearly linearly independent so they span a basis of dimension 3.
- ▶ Rank-Nullity Theorem shows that the null space has dimension 3 so these columns are a basis for null space.



Lecture 18 - Recap

Theorems & Lemmas

Fire Engine Lemma

Projection of
b along a

Project_along

Pythagorean theorem

Concept & Definition

Inner Product

Vector Norm

Orthogonality

Previous

1D Fire
engine
problem

Tools



Thank You For Your Attention!





Exercise: Orthogonality

- 1) Let $u = [-1, 2, 2]$ and $v = [1, 1, 1]$ be vectors. Determine if they are orthogonal

No, since $\langle u, v \rangle = -1 + 2 + 2 = 3 \neq 0$

- 2) Calculate the distance between u and v

$$\|u - v\| = \|[-2, 1, 1]\| = \sqrt{\langle [-2, 1, 1], [-2, 1, 1] \rangle} = \sqrt{(-2)^2 + 1 + 1} = \sqrt{6}$$

- 3) Find a vector v^* that is the orthogonal projection of v onto u

First calculate sigma: $\sigma_{vu} = \frac{\langle v, u \rangle}{\langle u, u \rangle} = \frac{3}{1+4+4} = \frac{1}{3}$. Then $v^{\perp u} = v - \frac{1}{3}u = [\frac{4}{3}, \frac{1}{3}, \frac{1}{3}]$

- 4) Calculate the distance of $w = [0, 3, 3]$ from $\text{span}\{u, v\} = \text{span}\{u, v^*\}$

First, we find $w^{\perp \text{span}\{u, v\}}$ by calculating the sigmas: $\sigma_{wu} = \frac{\langle w, u \rangle}{\langle u, u \rangle} = \frac{0+6+6}{9} = \frac{4}{3}$

and $\sigma_{wv^*} = \frac{\langle w, v^* \rangle}{\langle v^*, v^* \rangle} = \frac{0+1+1}{\frac{16}{9} + \frac{1}{9} + \frac{1}{9}} = \frac{2}{2} = 1$. Then we get

$w^{\perp \text{span}\{u, v\}} = w - \frac{4}{3}u - 1v = [0, 0, 0]$ so w is in $\text{span}\{u, v\}$ and its distance is 0

- 5) Find a basis for $\text{span}\{u, v, w\}$ and how would you find a basis for the null space?

Since w is in $\text{span}\{u, v\}$ but they could be orthogonalized, $\{u, v\}$ forms a basis for $\text{span}\{u, v, w\}$.

To find a basis for the null space, we look at the matrix of sigmas:
$$\begin{bmatrix} 1 & \sigma_{vu} & \sigma_{wu} \\ 0 & 1 & \sigma_{wv^*} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & \frac{1}{3} & \frac{4}{3} \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

And take the third column vector of its transpose (highlighted, corresponding to w) which is $[0, 0, 1]$

Wrong! We need to find the inverse, not the transpose. This takes a few more steps so let's skip it